

TOOL SPECIFICATION

Personal Productivity & Task Execution Agent — 13 tools

Prepared by: Hooria · AI Summer Internship — NLP & AI Agents Track · July 2026

Every tool follows the same contract: input is validated against a fixed schema before anything runs; execution never raises — it always returns a result envelope of the shape { success, data, error }. Field names, types, and JSON shapes below are exact and sufficient to reimplement each tool independently of the reference source. “Read” tools only fetch data; “Write” tools change stored data or produce an outward-facing artifact. * extract_meeting_actions itself does not require approval because it only reads/derives text — the task-creation step that typically follows it is what's gated.

create_task

Creates a single new task in the task store. Used for straightforward task creation requests such as “add a task to review the budget by Friday.”

Operation: Write	Approval required: No	Backing store: tasks table
-------------------------	------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
title	string	Required	Task title. 1-200 characters.
description	string	Optional	Free-text detail. Defaults to empty string.
priority	enum: Low Medium High Critical	Optional	Defaults to Medium.
due_date	date (ISO 8601, YYYY-MM-DD)	Optional	Omit if there is no deadline.
tags	array of string	Optional	Defaults to an empty list.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	False if validation or storage failed.
task	object (Task)	Yes, on success	The full created task record, including generated task_id, status (defaults to Pending), created_date, updated_date.
task_id	string	Yes, on success	Convenience copy of task.task_id, format task_<10 hex chars>.
confirmation	string	Yes, on success	Human-readable confirmation message.

Field	Type	Always present?	Description
error	string	Yes, on failure	Human-readable reason for failure.

Possible errors

- Invalid arguments — title missing or empty (schema validation fails before the tool runs).
- Database/storage failure — the write could not be committed; returned as success=false with a descriptive error, never as a raw exception.

Example call

```
{
  "tool": "create_task",
  "args": {
    "title": "Review Q3 budget",
    "description": "Check spend against forecast before Friday's meeting",
    "priority": "High",
    "due_date": "2026-07-24",
    "tags": ["finance"]
  }
}
```

Example result

```
{
  "success": true,
  "data": {
    "task": {
      "task_id": "task_9f2a1c7e0b",
      "title": "Review Q3 budget",
      "priority": "High",
      "status": "Pending",
      "due_date": "2026-07-24",
      "tags": ["finance"],
      "created_date": "2026-07-17T09:02:11Z",
      "updated_date": "2026-07-17T09:02:11Z"
    },
    "task_id": "task_9f2a1c7e0b",
    "confirmation": "Task 'Review Q3 budget' created."
  }
}
```

list_tasks

Retrieves tasks from the store, optionally filtered by status, priority, due date, or tag. Used for any “show me my tasks” style request.

Operation: Read	Approval required: No	Backing store: tasks table
------------------------	------------------------------	-----------------------------------

Input schema

Field	Type	Required?	Description
status	enum: Pending In Progress Blocked Completed Cancelled	Optional	Omit to include all statuses.

Field	Type	Required?	Description
priority	enum: Low Medium High Critical	Optional	Omit to include all priorities.
due_before	date	Optional	Only tasks due on or before this date.
tag	string	Optional	Only tasks that include this tag.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
tasks	array of Task	Yes, on success	Matching tasks, sorted by due date ascending (tasks with no due date last).
total_count	integer	Yes, on success	Number of matching tasks.

Possible errors

- Database/storage failure while querying — returned as success=false.

Example call

```
{
  "tool": "list_tasks",
  "args": { "status": "Pending", "priority": "Critical" }
}
```

Example result

```
{
  "success": true,
  "data": {
    "tasks": [
      { "task_id": "task_ab12cd34ef", "title": "Fix login bug",
        "priority": "Critical", "status": "Pending", "due_date": "2026-07-18" }
    ],
    "total_count": 1
  }
}
```

update_task

Modifies one or more fields on an existing task. Gated: never executes without explicit human approval.

Operation: Write	Approval required: Yes	Backing store: tasks table
-------------------------	-------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
task_id	string	Required	Must reference an existing task.
title	string	Optional	New title.
description	string	Optional	New description.

Field	Type	Required?	Description
priority	enum: Low Medium High Critical	Optional	New priority.
due_date	date	Optional	New due date.
status	enum: Pending In Progress Blocked Completed Cancelled	Optional	New status.
tags	array of string	Optional	Replaces the existing tag list.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
task	object (Task)	Yes, on success	The task after the update, with updated_date refreshed.
error	string	Yes, on failure	e.g. "Unknown task ID"

Possible errors

- Unknown task ID — no task with that ID exists; returned as success=false, never a raw exception.
- Invalid arguments — an unsupported enum value for priority/status.
- Database/storage failure while writing the update.

Example call

```
{
  "tool": "update_task",
  "args": { "task_id": "task_ab12cd34ef", "status": "In Progress" }
}
```

Example result

```
{
  "success": true,
  "data": {
    "task": { "task_id": "task_ab12cd34ef", "title": "Fix login bug",
              "status": "In Progress", "updated_date": "2026-07-17T10:14:02Z" }
  }
}
```

complete_task

Marks a task as Completed and stamps the completion time. Gated: requires explicit human approval.

Operation: Write	Approval required: Yes	Backing store: tasks table
-------------------------	-------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
task_id	string	Required	Must reference an existing task.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
task	object (Task)	Yes, on success	The task with status set to Completed.
status	string	Yes, on success	Always “Completed” on success.
completion_timestamp	string (ISO 8601 datetime)	Yes, on success	UTC time the completion was recorded.
error	string	Yes, on failure	e.g. “Unknown task ID”

Possible errors

- Unknown task ID — no task with that ID exists.
- Database/storage failure while writing the update.

Example call

```
{
  "tool": "complete_task",
  "args": { "task_id": "task_ab12cd34ef" }
}
```

Example result

```
{
  "success": true,
  "data": {
    "task": { "task_id": "task_ab12cd34ef", "status": "Completed" },
    "status": "Completed",
    "completion_timestamp": "2026-07-17T10:20:45Z"
  }
}
```

search_notes

Keyword search over saved notes, with an optional category filter and date range, returning results ranked by a relevance score.

Operation: Read	Approval required: No	Backing store: notes table
------------------------	------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
query	string	Required	1+ characters. Matched against title, content, and tags.
category	string	Optional	Restrict to one category.

Field	Type	Required?	Description
date_from	date	Optional	Only notes created on/after this date.
date_to	date	Optional	Only notes created on/before this date.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
results	array of Note + match_score	Yes, on success	Each item is a note object plus a match_score (0-1 float), sorted descending by score.
total_count	integer	Yes, on success	Number of matching notes.

Possible errors

- Invalid arguments — empty query string.
- Database/storage failure while searching.

Example call

```
{
  "tool": "search_notes",
  "args": { "query": "marketing campaign" }
}
```

Example result

```
{
  "success": true,
  "data": {
    "results": [
      { "note_id": "note_11a2b3c4d5", "title": "Marketing campaign kickoff",
        "match_score": 0.92 }
    ],
    "total_count": 1
  }
}
```

save_note

Saves a new note to the notes store, e.g. from meeting minutes or a freeform observation the user wants to keep.

Operation: Write	Approval required: No	Backing store: notes table
-------------------------	------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
title	string	Required	1-200 characters.
content	string	Optional	Defaults to empty string.
category	string	Optional	Defaults to “general”.
tags	array of	Optional	Defaults to an empty list.

Field	Type	Required?	Description
	string		

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
note_id	string	Yes, on success	Format note_<10 hex chars>.
confirmation	string	Yes, on success	Human-readable confirmation message.

Possible errors

- Invalid arguments — title missing or empty.
- Database/storage failure while writing.

Example call

```
{
  "tool": "save_note",
  "args": { "title": "Client call notes", "content": "Discussed timeline slip.",
            "category": "clients" }
}
```

Example result

```
{
  "success": true,
  "data": { "note_id": "note_77bb22cc11", "confirmation": "Note 'Client call notes'
saved." }
}
```

extract_meeting_actions

Sends meeting notes to the LLM with a fixed extraction prompt and returns a structured summary, decisions, action items (with owner/deadline where stated), and unresolved questions. Read-only — does not write to the store by itself.

Operation: Read (LLM-backed)	Approval required: No*	Backing store: none (LLM call only)
-------------------------------------	-------------------------------	-------------------------------------

Input schema

Field	Type	Required?	Description
transcript	string	Required	Raw meeting notes or transcript text, 1+ characters.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
summary	string	Yes, on	One-paragraph summary.

Field	Type	Always present?	Description
		success	
decisions	array of string	Yes, on success	Decisions made during the meeting.
action_items	array of object	Yes, on success	Each item: { task: string, owner: string null, deadline: date string null }.
unresolved_questions	array of string	Yes, on success	Open questions the meeting did not resolve.

Possible errors

- LLM API error — the model provider request failed (network, quota, auth).
- Invalid model response — the model did not return valid JSON matching the expected schema.

Example call

```
{
  "tool": "extract_meeting_actions",
  "args": { "transcript": "Team sync: launch delayed one week. Ali will update the landing page by Friday. Sara will fix the login bug." }
}
```

Example result

```
{
  "success": true,
  "data": {
    "summary": "The team decided to delay the launch by one week...",
    "decisions": ["Launch delayed to next week"],
    "action_items": [
      { "task": "Update the landing page", "owner": "Ali", "deadline": "2026-07-24" },
      { "task": "Fix the login bug", "owner": "Sara", "deadline": null }
    ],
    "unresolved_questions": []
  }
}
```

generate_work_plan

Builds an ordered daily schedule from currently pending/in-progress/blocked tasks, weighted by priority, deadline, and status, within a stated hours budget.

Operation: Read / Compute	Approval required: No	Backing store: tasks table (read-only)
----------------------------------	------------------------------	--

Input schema

Field	Type	Required?	Description
available_hours	number (float)	Required	Greater than 0 and at most 24.
plan_date	date	Optional	Defaults to today.
priorities	array of	Optional	User-stated focus areas; defaults to the top 3 scheduled

Field	Type	Required?	Description
	string		tasks if omitted.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
date	date string	Yes, on success	The plan date used.
ordered_schedule	array of object	Yes, on success	Each item: { task_id, title, priority, estimated_hours, due_date }.
recommended_focus_areas	array of string	Yes, on success	—
deferred_tasks	array of string (task_id)	Yes, on success	Tasks that did not fit in the available hours, or are blocked.
risk_warnings	array of string	Yes, on success	e.g. overdue or blocked tasks.
remaining_hours	number (float)	Yes, on success	Hours left unscheduled.

Possible errors

- Invalid arguments — available_hours out of range (≤ 0 or > 24).
- Database/storage failure while reading tasks.

Example call

```
{
  "tool": "generate_work_plan",
  "args": { "available_hours": 6 }
}
```

Example result

```
{
  "success": true,
  "data": {
    "date": "2026-07-17",
    "ordered_schedule": [
      { "task_id": "task_ab12cd34ef", "title": "Fix login bug", "priority":
"Critical",
        "estimated_hours": 2.0, "due_date": "2026-07-18" }
    ],
    "recommended_focus_areas": ["Fix login bug"],
    "deferred_tasks": [],
    "risk_warnings": [],
    "remaining_hours": 4.0
  }
}
```

detect_overdue_tasks

Finds tasks whose due date has passed and whose status is not Completed or Cancelled.

Operation: Read	Approval required: No	Backing store: tasks table (read-only)
------------------------	------------------------------	--

Input schema

Field	Type	Required?	Description
as_of	date	Optional	Defaults to today.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
overdue_tasks	array of Task	Yes, on success	—
total_count	integer	Yes, on success	—

Possible errors

- Database/storage failure while querying.

Example call

```
{
  "tool": "detect_overdue_tasks",
  "args": {}
}
```

Example result

```
{
  "success": true,
  "data": {
    "overdue_tasks": [
      { "task_id": "task_66dd11ee22", "title": "Review marketing metrics",
        "due_date": "2026-07-16" }
    ],
    "total_count": 1
  }
}
```

create_reminder

Creates a standing reminder (stored as a lightweight task tagged “reminder”). Gated: requires explicit human approval.

Operation: Write	Approval required: Yes	Backing store: tasks table
-------------------------	-------------------------------	----------------------------

Input schema

Field	Type	Required?	Description
title	string	Required	Short reminder label.
due_date	date	Required	When the reminder is due.

Field	Type	Required?	Description
note	string	Optional	Extra detail. Defaults to empty string.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
reminder_id	string	Yes, on success	The underlying task_id.
confirmation	string	Yes, on success	Human-readable confirmation.

Possible errors

- Invalid arguments — missing title or due_date.
- Database/storage failure while writing.

Example call

```
{
  "tool": "create_reminder",
  "args": { "title": "Project review", "due_date": "2026-07-21" }
}
```

Example result

```
{
  "success": true,
  "data": { "reminder_id": "task_44ff55aa66", "confirmation": "Reminder set for 2026-07-21." }
}
```

summarize_notes

Retrieves recent notes (optionally filtered by category) so the agent can summarize them in its response. Does not call the LLM itself — it only fetches the source material.

Operation: Read	Approval required: No	Backing store: notes table (read-only)
-----------------	-----------------------	--

Input schema

Field	Type	Required?	Description
category	string	Optional	Restrict to one category.
max_notes	integer	Optional	Defaults to 10.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
notes	array of Note	Yes, on	Most recent notes first.

Field	Type	Always present?	Description
		success	
total_count	integer	Yes, on success	Number of notes returned.

Possible errors

- Database/storage failure while querying.

Example call

```
{
  "tool": "summarize_notes",
  "args": { "category": "marketing", "max_notes": 5 }
}
```

Example result

```
{
  "success": true,
  "data": {
    "notes": [{ "note_id": "note_11a2b3c4d5", "title": "Marketing campaign kickoff"
  }],
    "total_count": 1
  }
}
```

draft_followup_email

Drafts a follow-up email based on meeting notes using the LLM. The email is never actually sent — this always produces a draft only. Gated: requires explicit human approval.

Operation: Write (simulated / LLM-backed)	Approval required: Yes	Backing store: none (LLM call only)
--	-------------------------------	-------------------------------------

Input schema

Field	Type	Required?	Description
meeting_notes	string	Required	Source notes to draft from.
recipient_name	string	Optional	Defaults to “the team”.
tone	string	Optional	Defaults to “professional”.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
subject	string	Yes, on success	Drafted subject line.
body	string	Yes, on success	Drafted email body.

Field	Type	Always present?	Description
note	string	Yes, on success	Always states the email was drafted, not sent.

Possible errors

- LLM API error — the model provider request failed.
- Invalid model response — the model did not return valid JSON.

Example call

```
{
  "tool": "draft_followup_email",
  "args": { "meeting_notes": "Discussed Q3 roadmap...", "recipient_name": "Ali" }
}
```

Example result

```
{
  "success": true,
  "data": {
    "subject": "Follow-up: Q3 Roadmap Discussion",
    "body": "Hi Ali, thanks for the discussion today...",
    "note": "This email was drafted, not sent. Sending is not implemented (simulation only).",
  }
}
```

generate_weekly_report

Aggregates the past week's completed, overdue, and blocked tasks, and recommends priorities for the following week.

Operation: Read / Compute	Approval required: No	Backing store: tasks table (read-only)
----------------------------------	------------------------------	---

Input schema

Field	Type	Required?	Description
week_ending	date	Optional	Defaults to today; the report covers the 7 days up to and including this date.

Output schema

Field	Type	Always present?	Description
success	boolean	Yes	—
week_start	date string	Yes, on success	—
week_ending	date string	Yes, on success	—
completed_count	integer	Yes, on success	—
completed_tasks	array of	Yes, on	Titles of tasks completed this week.

Field	Type	Always present?	Description
	string	success	
overdue_count	integer	Yes, on success	—
overdue_tasks	array of string	Yes, on success	Titles of currently overdue tasks.
blocked_count	integer	Yes, on success	—
blocked_tasks	array of string	Yes, on success	Titles of currently blocked tasks.
recommended_next_week_priorities	array of string	Yes, on success	Top 5 pending/in-progress tasks by priority and due date.

Possible errors

- Database/storage failure while querying.

Example call

```
{
  "tool": "generate_weekly_report",
  "args": {}
}
```

Example result

```
{
  "success": true,
  "data": {
    "week_start": "2026-07-10", "week_ending": "2026-07-17",
    "completed_count": 3, "completed_tasks": ["Send onboarding email"],
    "overdue_count": 1, "overdue_tasks": ["Review marketing metrics"],
    "blocked_count": 1, "blocked_tasks": ["Write unit tests for payments module"],
    "recommended_next_week_priorities": ["Fix login bug", "Prepare investor update slides"]
  }
}
```